



MANIPAL INSTITUTE OF TECHNOLOGY

MANIPAL

(A constituent unit of MAHE, Manipal)

SCHOOL OF COMPUTER ENGINEERING

CERTIFICATE

This is to certify that Ms./Mr.

Reg. No.: Section:..... has satisfactorily completed the lab exercises prescribed for Database Systems Lab [MCA 4141] of First Semester M.C.A. (School of Computer Engineering) Degree at MIT, Manipal, in the academic year 2025-2026.

Date:

Signature

Faculty in Charge

Course Outcomes (COs)

<i>At the end of this course, the student should be able to:</i>		No. of Contact Hours
C01:	Create and modify database objects.	6
C02:	Manipulate the data in the database.	3
C03:	Design queries to retrieve the data from database.	9
C04:	Perform database operations by integrating procedural language constructs, stored programs.	9
C05:	Develop database application.	6

Evaluation Pattern

Continuous Evaluation	60%
Internal Lab Evaluations	Mini Project- 16(4+12) Mid Term Exam- 20 Observation Book- 6*2=12 Execution- 6*2=12 Max.Marks:60
Final Examination	40%
End Sem Lab Exam	Examination of 3 hours duration that includes questions based on: SQL + PL/SQL Max.Marks:40

INSTRUCTIONS TO STUDENTS

1. Students should be regular and come prepared for the lab practice.
2. In case a student misses a class, it is his/her responsibility to complete that missed exercise(s).
3. Students should bring the observation book daily for the lab.
4. They should implement the given query/program individually.
5. Students should clearly listen to the instructions given by the faculty.

6. Once the query/program gets executed, they should show the query/program and results to the instructors and copy the same in their observation book.
7. When copying down the query in the observation book the template to be followed is:
 - a. Question No
 - b. Question
 - c. Query statement
 - d. Output
8. When copying down the PL/SQL program in the observation book the template to be followed is:
 - a. Program No
 - b. Program title
 - c. Program Code
 - d. Output
9. Questions for lab tests and exam need not necessarily be limited to the questions in the manual, but could involve some variations and / or combinations of the questions.

CONTENTS

Lab	Topics to be covered	Page No	Signature
1-2	SQL Basics with DDL Commands-CREATE, ALTER, DROP		
3	Populate and manipulate the database using DML Commands -INSERT, UPDATE, DELETE		
4-5	Mini Project Abstract submission SQL Simple, join, subquery and Advanced Queries		
6	Mid Term Practical Examination		
7	Requirement specification submission PL/SQL and Cursors .		
8	Exception Handling and Triggers		
9	Schema diagram & Schema design Procedures and Functions		
10	Mini-Project Implementation		
11	Mini Project Demo.		
12	End Term Practical Examination		

DATABASE MANAGEMENT SYSTEMS LABORATORY MANUAL- MCA 4141

Relational Schema

This manual is based on the following Company Database schema (derived from the ER diagram):

DEPT(DNO, DNAME, LOCATION)

EMP (EMPNO, ENAME, JOB, MGRID, DATE_BIRTH, SAL, COMM, DEPTNO, PRJ_ID, DATE_JOIN)

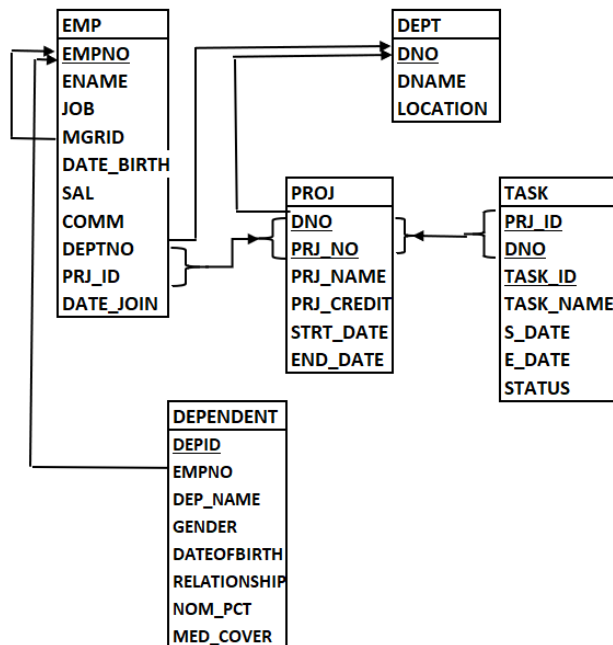
PROJ(DNO, PRJ_NO, PRJ_NAME, PRJ_CREDIT, STRT_DATE, END_DATE)

TASK (PRJ_ID, DNO, TASK_ID, TASK_NAME, S_DATE, E_DATE, STATUS)

DEPENDENT(DEPID, EMPNO, DEP_NAME, GENDER, DATEOFBIRTH, RELATIONSHIP, NOM_PCT, MED_COVER)

Relationships

Schema Diagram



- DEPT (DNO) is the parent of EMP (DEPTNO) — an employee works in one department.
- EMP (EMPNO) is self-referenced by EMP (MGRID) — an employee's manager is also an employee.
- DEPT (DNO) is the parent of PROJ (DNO) — a department controls many projects (identifying relationship; PROJ's primary key is composite: DNO + PRJ_NO).

- PROJ (PRJ_NO, DNO) is the parent of TASK (PRJ_ID, DNO) — a project is broken into many tasks (identifying relationship; TASK's primary key is composite: PRJ_ID + DNO + TASK_ID).
 - EMP (EMPNO) is the parent of DEPENDENT (EMPNO) — an employee can have many dependents.
 - EMP (PRJ_ID) references PROJ (PRJ_NO) — an employee is assigned to a project.
-

LAB 1: DDL COMMANDS — TABLE CREATION WITH CONSTRAINTS

1.1 Objective

To design and create the tables of the Company database using CREATE TABLE, applying appropriate data types and constraints (PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, CHECK, DEFAULT), and to practice ALTER, DROP, TRUNCATE, RENAME.

1.2 SQL BASICS

SQL (Structured Query Language) is a database language designed for the database schema creation and modification, retrieval and management of data in relational database management systems (RDBMS).

Various Data Types :

1. Character

- a. Char – fixed length character string that can varies between 1-2000 bytes
- b. Varchar / Varchar2 – variable length character string, size ranges from 1-4000 bytes.it saves the disk space
- c. Long - variable length character string, maximum size is 2 GB

2. Number : Can store +ve,-ve,zero,fixed point,floating point with 38 precission.

- a. Number – {p=38,s=0}
- b. Number(p) - fixed point
- c. Number(p,s) –floating point (p=1 to 38,s= -84 to 127)

3. Date : used to store date and time in the table. DB uses its own format of storing in fixed length of 7 bytes for century, date, month, year, hour, minutes, seconds. The default data type is “dd-mon-yy”

4. Raw Datatype: used to store byte oriented data like binary data and byte string.

5. Other :

- a. CLOB – stores character object with single byte character.
- b. BLOB – stores large binary objects such as graphics,video,sounds.
- c. BFILE – stores file pointers to the LOB's.

Different types of commands in SQL:

- 1) **DDL commands:** - To create a database objects
- 2) **DML commands:** - To manipulate data of a database objects.
- 3) **DQL command:** - To retrieve the data from a database.
- 4) **DCL commands:** - To control the data of a database.
- 5) **TCL commands:-** To control and manage transactions.

DDL commands:

1. **The Create Table Command:** It defines each column of the table uniquely. Each column has minimum of three attributes, a name, data type and size.

Syntax:

Create table <table name>
(<col1> <datatype>(<size>), <col2> <datatype>(<size>),...);

Data constraints:

Two Types- Table Level and Column Level

a) Not null constraint at column level

Syntax:

<col> <datatype>(<size>) **not null**

b) Unique constraint at column level

Syntax:

<col> <datatype>(<size>) **unique**

c) Unique constraint at table level

Syntax:

Create table tablename (<col1> <datatype>(<size>), <col2>
<datatype>(<size>), **unique** (<col1>, <col2>));

d) Primary key constraint at column level

Syntax:

<col> <datatype>(<size>) **primary key**;

e) Primary key constraint at table level

Syntax:

Create table tablename (<col1> <datatype>(<size>), <col2>
<datatype>(<size>), **primary key**(<col1>, <col2>));

f) Foreign key constraint at column level

Syntax:

<col><datatype>(<size>) **references** <tablename>[<col>];

g) Foreign key constraint at table level

Syntax:

Foreign key (<col1>[, <col2>]) **references** <tablename>(<col1>, <col2>));

h) Check constraint at column level

Syntax:

<col> <datatype>(<size>) **check**(<logical expression>)

i) Check constraint at table level

Syntax:

Check (<logical expression>)

Naming constraints with **CONSTRAINT constraint_name** ... makes them easier to **ALTER** ... **DROP CONSTRAINT** later; unnamed constraints get a system-generated name like **SYS_C00123**

2. Modifying the structure of tables

a) Adding new columns

Syntax:

Alter table <tablename>
add(<new col> <datatype>(<size>),<new col> <datatype>(<size>),...);

b) Dropping a column from a table

Syntax:

Alter table <tablename> **drop column** <col>;

c) Modifying existing columns

Syntax:

Alter table <tablename> **modify** (<col> <newdatatype>(<newsizesize>));

3. Renaming the tables

Syntax:

Rename <oldtable> to <new table>;

4. Truncating Tables: It empties a table completely.

Syntax:

Truncate table <tablename>;

5. Destroying tables.

Syntax:

Drop table <tablename>;

1.3 Table Structure

Table	Column	Data Type	Constraint	Constraint_Name
DEPT	DNO	CHAR(3)	Unique and starts from 'D'	C_DNO_UQ and C_D_UQ
	DNAME	VARCHAR(15)	NOT NULL, UNIQUE	C_DNAME_UQ
	LOCATION	CHAR(3)	(MUB,HYD,BNG,MNG,CHN)	C_LOC
EMP	EMPNO	NUMBER(3)	PRIMARY KEY	
	ENAME	VARCHAR(15)	NOT NULL	
	JOB	VARCHAR(15)	(MGR,A.MGR,CLK,DEV,ENG,CEO,GM)	C_JOBCHK
	MGRID		REFERENCES EMP	C_FK_MGRID
	DATE_BIRTH	Date	Less Than Joining Date	C_DB
	SAL	NUMBER(10,2)	Greater than 20000	C_SALCHK
	COMM	NUMBER(10,2)	DEFAULT 1000	
	DEPTNO	CHAR(3)	REFERENCES DEPT ON DELETE CASCADE	
	DATE_JOIN	DATE	SHOULD BE GREATER THAN DATE_BIRTH	

Table	Column	Data Type	Constraint	Constraint_Name
PROJ	DNO		REFERENCES DEPT	C_FKDEPT
	PRJ_NO	CHAR(3)	Starts from P and NOT NULL	C_PRJNO
	PRJ_NAME	VARCHAR(20)	NOT NULL	
	PRJ_CREDIT	NUMBER(2)	SHOULD BE GREATER THAN 0 AND LESS THAN 11	C_PCREDIT
	STRT_DATE	DATE	NOT NULL	
	END_DATE	DATE	END_DATE should be greater than STRT_DATE	C_ENDATE
TASK	PRJ_ID	CHAR(3)	REFERENCES PROJ ON DELETE SET NULL	
	DNO	CHAR(3)		
	TASK_ID	NUMBER(3)		
	TASK_NAME	VARCHAR(30)	NOT NULL	
	S_DATE	DATE	NOT NULL	
	E_DATE	DATE	SHOULD BE GREATER THAN S_DATE	C_TEDATE
	STATUS	VARCHAR(12)		
DEPENDENT	DEPID	NUMBER(2)	PRIMARY KEY	
	EMPNO		REFERENCES EMP	
	DEPENDENT_NAME	VARCHAR(20)	NOT NULL	
	GENDER	CHAR(1)	(M,F)	
	DOFBIRTH	DATE	NOT NULL	
	RELATIONSHIP	VARCHAR2(15)	(SPOUSE,CHILD,PARENT)	C_CHKREL
	NOM_PCT	NUMBER(5,2)	NOM_PCT BETWEEN 0 AND 100	C_CHKNPCT
	MED_COVER	VARCHAR2(3)	(YES,NO) DEFAULT 'NO'	

LAB 2: ALTER TABLE

1. Make the combination of **DNO and PRJ_NO** as primary key in the table PROJ.
2. Add a column to EMP table named **PROJ_ID**. Add a foreign key constraint to EMP table on **(DeptNo, Proj_Id)** referencing **PROJ**. (Indicates an employee from which department is working on which project/s.)
3. Add constraints(VALID_EMPNO_Cons) to the EMP table to check the EMPNO >100.
4. Add a new column **Dept_Budget** column of size 7 digits to the DEPT table.
5. Add a new column **Prj_Fund** column of size 7 digits to the PROJ table.
6. Add constraint to **STATUS** column of TASK table to include values {PENDING, IN-PROGRESS, COMPLETED}.
7. In the table TASK, make Task_Id,DNO,Prj_ID as the primary key.
8. In EMP, add a column BONUS with data type NUMBER(8,2) with default value as 0.
9. In EMP, change the column BONUS datatype as Number(6,2), change the name of the column as BONUSSES and remove that column

LAB 3: DML COMMANDS — DATA INSERTION, UPDATE, DELETE

3.1 Objective

To populate the Company database tables with sample data using INSERT, and to practice UPDATE and DELETE.

3.2 Notes

Inserting Data into Tables: Once a table is created the most natural thing to do is load this table with data to be manipulated later.

Syntax:

Insert into <tablename> (<col1>, <col2>) **values** (<exp1>,<exp2>);

Delete operations

a) Remove all rows

Syntax:

Delete from <tablename>;

b) Removal of specific rows

Syntax:

Delete from <tablename> **where** <condition>;

Updating the contents of a table.

a) Updating all rows

Syntax:

Update <tablename> **set** <col1> = <exp1>, <col2> = <exp2>;

b) Updating seleted records

Syntax:

Update <tablename> **set** <col1>=<exp1>, <col2>=<exp2>
where <condition>;

TCL Commands

Commit: A COMMIT ends the current transaction and makes permanent any changes made during the transaction.

Syntax:

COMMIT;

ROLLBACK : It ends the transaction but undoes any changes made during the transaction.

Syntax:

ROLLBACK [WORK] [TO SAVEPOINT]

SAVEPOINT: It marks and saves the current point in the processing of a transaction.

Syntax:

SAVEPOINT <SavePointName>

2.3 Insert Data

- A student can enter a valid value into the columns left blank, columns with *null* must be entered with NULL value only.

- If some records cannot be inserted because of violation of constraints, then write the reason for violation in the lab book and choose a data satisfying the constraint and insert them into the table.
- **Insert 2 records of your own for each table**
- **Give Commit after insertion**

DEPT

DNO	DName	Location	Dept_Budget
D1	Marketing	CHN	500000
D2	Research	MNG	300000
D3	IT	BNG	400000
D4	HR	BGG	200000
D5	Accounts	BNG	500000
Null	Corporate	HYD	700000

EMP

Empno	Ename	Job	MgR ID	Date_Birth	Sal	Comm	Deptno	Prj_Id	Date_Join
100	Ravi	MGR	111	10-10-1995	32000		D1	P1	2-10-2006
102	Raviraj	ENG	106	10-01-1985	24000		D1	P3	12-11-2010
111	Raghu	ENG	150	10-12-1984	45000	15000	null	null	3-02-2005
150		CEO	null	10-12-1990	60000	30000	null	null	3-12-2008
103		DEV	111	10-12-1980			D1	P1	22-09-2001
103		CLRK	111	2-08-1980			D1	P3	12-10-2002
125	Manu	DEV	150	10-12-1980			D4	P3	2-10-2002
104		CLERK	125	2-10-1980			D2	P1	26-11-2005
106		A.MGR	111	2-10-1986			D2		24-11-1985
123	Mahesh	CLRK	150	10-12-1984	25000		D3	P2	23-12-2008
108		DEV	106	10-11-1980			D9		2-10-2005
103		DEV	111	10-07-1980			D1	P3	21-06-2001
null		CLRK	106	10-01-1985	18000		D5		10-12-2007

PROJ

DEPT_NO	PROJ_NO	PROJ_NAME	Prj_Credit	BUDGET	START_DATE	END_DATE
D1	P1	PAYROLL AUTOMATION	4	500000	01-JAN-2022	30-JUN-2022

D2	P1	AI RESEARCH PLATFORM	7	900000	15-FEB-2022	14-FEB-2023
D3	P2	DATA WAREHOUSE MIGR.	8	650000	01-MAR-2022	31-AUG-2022
D4	P3	CRM REVAMP	5	400000	10-APR-2022	09-OCT-2022
D1	P3	ERP REVAMP	9	500000	11-JAN-2020	12-DEC-2021
D5	P4	WAREHOUSE AUTOMATION	10	700000	05-MAY-2022	04-NOV-2022

TASK

PROJ_NO	DEPT_NO	TASK_NO	TASK_NAME	START_DATE	END_DATE	STATUS
P1	D1	1	REQUIREMENT ANALYSIS	01-JAN-2022	15-JAN-2022	COMPLETED
P1	D2	2	MODULE DESIGN	16-JAN-2022	10-FEB-2022	COMPLETED
P2	D3	3	CODING & TESTING	11-FEB-2022	NULL	IN-PROGRESS
P3	D4	1	DATASET COLLECTION	15-FEB-2022	28-FEB-2022	COMPLETED
P3	D1	2	MODEL TRAINING	01-MAR-2022	NULL	IN-PROGRESS
P3	D1	1	SCHEMA MAPPING	01-MAR-2022	20-MAR-2022	COMPLETED
P4	D5	1	REQUIREMENT GATHERING	10-APR-2022	NULL	PENDING
P4	D5	2	SITE SURVEY	05-MAY-2022	20-MAY-2022	COMPLETED

DEPENDENT

DEP_ID	EMP_ID	NAME	GENDER	DOB	RELATION	PERCENT_SHARE	INSURED
1	100	LATHA KUMAR	F	10-MAY-1982	SPOUSE	50	YES
2	100	ARUN KUMAR	M	19-JUN-2000	CHILD	25	YES
3	102	SEEMA RAVI	F	01-JAN-1980	SPOUSE	60	YES
4	103	MOHAN RAO	M	12-DEC-1950	PARENT	30	NO
5	106	NEHA RAHUL	F	22-JUL-1990	SPOUSE	70	YES
6	108	ROHAN MEERA	M	03-MAR-2015	CHILD	100	YES

2.4 UPDATE AND DELETE

1. Run **COMMIT** command. Delete the employee records working on project P3 and confirm the result. Type **ROLLBACK** to restore the records back if records are deleted.
2. Run **COMMIT** command. Delete *Accounts* department from the DEPT table and confirm the result with reason. Type **ROLLBACK** to restore the records back if records are deleted.
3. Run **COMMIT** command. Delete records of employees with Empno 125 and working in project P2. Type **ROLLBACK** to restore the records back, if records are deleted.
4. Run **COMMIT** command. Delete all the tasks under Project P2. If deleted, type ROLLBACK to restore the records.
5. Delete tasks that are still PENDING . If deleted, type **ROLLBACK** to restore the records.
6. Update the Job of employee with Empno 123 to MGR, salary to 35000 and his manager as 111.
7. Update the EMP table to set the default commission of all employees to Rs.10000/- who are working as managers.
8. Give a 10% raise to all ENGINEERs.

LAB 4: SIMPLE SQL QUERIES

4.1 Objective

To retrieve and filter data using SELECT, WHERE, DISTINCT, ORDER BY, logical/comparison operators, LIKE, BETWEEN, IN, and NULL handling.

4.2 Notes

Viewing data in the tables: Once data has been inserted into a table, the next most logical operation would be to view what has been inserted.

a) all rows and all columns

Syntax:

Select * from tablename;

b) Filtering table data: a method of filtering table data that is not required

Selected columns and all rows:

Syntax:

select <col1>,<col2> **from** <tablename>;

Selected rows and all columns:

Syntax:

select * from <tablename> **where** <condition>;

Selected columns and selected rows:

Syntax:

select <col1>,<col2> **from** <tablename> **where**<condition>;

Sorting data in the table

Syntax:

Select * from <tablename> **order by** <col1>,<col2> <[sortorder]>;

Conversion Functions

1. to_char() : This function converts date to a value of varchar type in a form specified by date format. If format is neglected then it converts date to varchar2 in the default date format.

Syntax: to_char(d,[format]);

2. to_date() : This function converts character to date data format specified in the form character.

Syntax: to_date(d,[format]);

Note:

- **Three-valued logic:** every comparison involving NULL (e.g. COMM = NULL, COMM <> NULL) evaluates to *unknown*, not true/false — this is why IS NULL / IS NOT NULL must be used instead of =/<>.
- **Wildcards in LIKE:** % matches zero or more characters, _ matches exactly one character. Use ESCAPE if the data itself contains % or _ literally.
- **DISTINCT** applies to the *entire* selected row combination, not to a single column in isolation, when multiple columns are listed.
- **BETWEEN is inclusive** of both boundary values; IN is a shorthand for multiple OR-ed equality checks.

4.3 Queries

1. Display records of Employees who have salary more than 25000 or working in department D1 .
2. List all employee with their names as "Employee Name" and their salaries as "SALARY", whose salary lies between 25200/- and 35200/- both inclusive
3. List the name of employees who is working at Locations (BNG,MUB,HYD) (using both OR , IN operator).
4. Display the records in the EMP table in the ascending order of Deptno and descending order of salary.
5. Display the task details for the project P3.
6. List out the employees who are not receiving commission.
7. Create a table Manager with columns Empno, Ename, Job, Deptno, Salary with structure and data copied from the EMP table.
8. List the Project Number, Project Name of all the projects handled by the department D2 and having project credits more than 5.
9. List all employees reporting to manager with empno 111.
10. Display name of employees whose 2nd & 3rd character is 'av',name length is 6 and ends with 'j'.
11. Display the task details for the task with the duration less than 1 week.
12. Find the name of the Projects starting after 1-Mar-2022 with credit above 5.
13. Find projects that have tasks starting after Feb 1, 2022.
14. List the Employees in department , ordered by salary descending
15. List the tasks that are still pending or in progress.
16. List the name of the female dependents with medical coverage

LAB 5: ADVANCED SQL QUERIES — AGGREGATE FUNCTIONS, GROUPING, JOIN, NESTED QUERIES AND BUILT-IN FUNCTIONS

5.1 Objective

To use aggregate (group) functions, GROUP BY, HAVING, JOIN and string/date/numeric functions.

5.2 Notes

An **aggregate** function returns a result based on group of rows. They are avg, max, min, sum and count

Group by clause

This allows us to use simultaneous column name and group functions.

Example: *Select max(percentage), deptname from student group by deptname;*

Having clause

This is used to specify conditions on rows retrieved by using group by clause.

Example: *Select max(percentage), deptname from student group by deptname having count(*)>=50;*

Order of evaluation: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY — this is why column aliases defined in SELECT normally can't be reused in WHERE

Special Operators with WHERE clause:

In / not in – used to select a equal value from a specific set of values

Any - used to compare with a specific set of values

Between / not between – used to find between the ranges

Like / not like – used to do the pattern matching

NESTED QUERIES AND JOINS

Nested Queries

Nesting of queries one within another is known as a nested queries.

Subqueries

The query within another is known as a subquery. A statement containing subquery is called parent statement. The rows returned by subquery are used by the parent statement

Example: *select ename, eno, address where salary > (select salary from employee Where ename ='jones');*

Relating Data through Join

The purpose of a join concept is to combine data spread across tables. A join is actually performed by the 'where' clause which combines specified rows of tables.

Syntax: select columns from table1, table2 where logical expression;

Simple Join

It is the most common type of join. It retrieves the rows from 2 tables having a common column and is further classified into

a) Equi-join

A join, which is based on equalities, is called equi-join.

Example: *select * from item, cust where item.id=cust.id; (old Syntax)*

select i.itemname, c.custname from item i join cust c on i.itemid = c.id;

select i.itemname, c.custname from item i natural join cust c; (if both primary key and foreign key have same names)

b) Non Equi-join

It specifies the relationship between columns belonging to different tables by making use of relational operators other than '='.

Example: *select * from item, cust where item.id < cust.id;*

Table Aliases

Table aliases are used to make multiple table queries shorter and more readable. We give an alias name to the table in the 'from' clause and use it instead of the name throughout the query.

Self join

Joining of a table to itself is known as self-join. It joins one row in a table to another. It can compare each row of the table to itself and also with other rows of the same table.

Example: *select * from emp x, emp y where x.salary >= (select avg(salary) from x.emp where x.deptno = y.deptno);*

Outer Join

It extends the result of a simple join. An outer join returns all the rows returned by simple join as well as those rows from one table that do not match any row from the table. The symbol(+) represents an outer join.

Example: *select ename, job, dname from emp, dept where emp.deptno (+) = dept.deptno;*

NOTE

- *Non-correlated subquery: runs once, independently of the outer query (e.g. finding the overall average salary). Correlated subquery: re-evaluates once per outer row because it references an outer column (e.g. E2.DEPTNO = E.DEPTNO), so it is generally slower on large tables.*
- *EXISTS vs IN: EXISTS just checks for the presence of any matching row and stops at the first hit (often faster, and safe with NULLs); IN builds and compares a full value list, and silently returns no rows if that list contains a NULL mixed with a NOT IN.*

- *ANY/ALL are not the same as MIN/MAX shortcuts in behaviour when the subquery can return zero rows: > ALL (empty set) is TRUE, while > ANY (empty set) is FALSE — always check whether the subquery could return no rows.*

- **SET OPERATORS & VIEWS IN RDBMS**

Set Operators:

The Set operator combines the result of 2 queries into a single result. The following are the operators:

- Union
- Union all
- Intersect
- Minus

Set operators require identical column count and compatible data types in both SELECT lists; column names/aliases are taken from the first query.

- *UNION removes duplicates (does an implicit sort/distinct — costs more); UNION ALL keeps duplicates and is faster when duplicates are known not to occur or don't matter.*
- *INTERSECT and MINUS are also duplicate-eliminating and require the same column signature across both queries.*

Views

A view is a “virtual table” or a “stored query” which takes the output of a query and treats it as a table. The table upon which a view is created is called as base table.

A view is a logical table based on a table or another view. A view contains no data of its own but is like a window through which data from tables can be viewed or changed.

Syntax:

Create [or replace] view <view name> [column alias names] as <query> [with <options> conditions];

Built in Functions in SQL

Single Row functions

A single row function or scalar function returns only one value for every row queries in table. Single row function can appear in a select command and can also be included in a where clause. The single row function can be broadly classified as,

- Date Function
- Numeric Function
- Character Function
- Conversion Function

Date Function

They operate on date values and produce outputs, which also belong to date data type except for months, between, date function returns a number.

1. Add_month: This function returns a date after adding a specified date with specified number of months.

Syntax: Add_months(d,n); where d-date n-number of months

2. last_day: It displays the last date of that month.

Syntax: last_day (d); where d-date

3. Months_between: It gives the difference in number of months between d1 & d2.

Syntax: month_between (d1,d2); where d1 & d2 -dates

4. next_day : It returns a day following the specified date.

Syntax: next_day (d,day);

5. round: This function returns the date, which is rounded to the unit specified by the format model.

Syntax : round (d,[fmt]); where d- date, [fmt] – optional. By default date will be rounded to the nearest day

Numerical Functions

Command	Query	Output
Abs(n)	Select abs(-15) from dual;	15
Ceil(n)	Select ceil(55.67) from dual;	56
Exp(n)	Select exp(4) from dual;	54.59
Floor(n)	Select floor(100.2) from dual;	100
Power(m,n)	Select power(4,2) from dual;	16
Mod(m,n)	Select mod(10,3) from dual;	1
Round(m,n)	Select round(100.256,2) from dual;	100.26
Trunc(m,n)	Select trunc(100.256,2) from dual;	100.23
Sqrt(m,n)	Select sqrt(16) from dual;	4

Character

Functions

Command	Query	Output
initcap(char);	<i>select initcap("hello") from dual;</i>	Hello

lower (char); upper (char);	<i>select lower ('HELLO') from dual;</i> <i>select upper ('hello') from dual;</i>	hello HELLO
ltrim (char,[set]);	<i>select ltrim ('cseit', 'cse') from dual;</i>	it
rtrim (char,[set]);	<i>select rtrim ('cseit', 'it') from dual;</i>	cse
replace (char,search string, replace string);	<i>select replace ('jack and jue', 'j', 'bl') from dual;</i>	black and blue
substr (char,m,n);	<i>select substr ('information', 3, 4) from dual;</i>	form

Queries :

1. List the total number of employees.
2. List the minimum, maximum and average salaries and rename the column as min_sal, max_sal, avg_sal, total salary of the employees.
3. List the Project names undertaken by Marketing Department.
4. Display the employees name in capital, lower, 1st character only capital, number of characters and 3 characters from 2nd position.
5. List the name of employees who are working under the manager 'Raghu'.
6. Display department name, Max salary and Min salary in each department.
7. Display number of employees working in each department and their department name.
8. List the employees whose experience is more than 5 years.
9. List the Employees who are born in the month of December and year 1984.
10. List the Number of employees per job title.
11. List the Departments with More Than Three Employees.
12. List the employees working under the project 'Payroll Automation'.
13. Write SQL query to illustrate ceiling, floor, truncate, and round functions on the value 2.83.

14. List the projects which have duration of more than 1 year.
15. List all employees along with the tasks they are associated with through their department's projects.
16. List the total project budget sanctioned per department.
17. List the count of tasks based on its status.
18. List Employee names in uppercase with joining year
19. List the Number of months each employee has served.
20. Display the name of Highest paid employee along with department name for every department.
21. List Job category and total salary paid for the each jobs category by the company.
22. Display name of the department from which maximum number of employees are working on project P1.(using subquery)
23. Select name and job of employees working either in 'Marketing' or 'Research' department.
24. Display name and salary of employees whose salary is greater than minimum salary of the company.(using subquery)
25. Display Employees Working on the Most Projects.
26. Display the Deptno of the department that has highest average salary of the company.(Subquery)
27. List the name of departments which are working on more than 1 project.(subquery)
28. Display how many employees joined after 15th of all months.
29. Write a query to list Employee number, name and Job of the employees who work in the same job as 'Mahesh'.
30. Create a View EMP_PRJ_VW to display records of employees of 'marketing' department and project in which they are working.
31. Display employee names and projects in which they are working using View EMP_PRJ_VW .
32. Calculate Total and Average Task Duration for Each Project
33. List out the number of employees joined in every month in ascending order.
34. Create an index on the columns (name and job) on EMP table.
35. Display the employees working in the same department as Ravi.

36. Display the name of the departments that have at least one project with budget above 400000 (correlated).
37. Display the names of employee who earn more than the average salary of their own department (correlated).
38. Display the name of Employees who have no dependents (NOT EXISTS)
39. List the name of developers or engineers.(SET OPERATION)

Lab 6 - MID TERM PRACTICAL EXAMINATION

Lab 7 - PL/SQL BASICS — CONTROL STRUCTURES

7.1 Objective

To write anonymous PL/SQL blocks using variables, %TYPE, %ROWTYPE, conditional statements, and loops.

7.2 NOTES

PL/SQL can process data using flow of statements. The flow of control statements are classified into the following categories.

- Conditional control -Branching
- Iterative control - looping
- Sequential control

BRANCHING in PL/SQL:

Sequence of statements can be executed on satisfying certain condition. If statements are being used and different forms of IF are:

Simple IF	IF-THEN-ELSE	ELSIF	NESTED IF
IF condition THEN statement1; statement2; END IF;	IF condition THEN statement1; ELSE statement2; END IF;	IF condition1 THEN statement1; ELSIF condition2 THEN statement2; ELSIF condition3 THEN statement3; ELSE Statement n; END IF;	IF condition THEN statement1; ELSE IF condition THEN statement2; ELSE statement3; END IF; END IF; ELSE statement3; END IF;

SELECTION IN PL/SQL(Sequential Controls)

SIMPLE CASE

Syntax:

CASE SELECTOR

WHEN Expr1 THEN statement1;

```
WHEN Expr2 THEN statement2;  
:  
ELSE  
Statement n;  
END CASE;
```

ITERATIONS IN PL/SQL

Sequence of statements can be executed any number of times using loop construct.

It is broadly classified into:

- Simple Loop
- For Loop
- While Loop

SIMPLE LOOP

Syntax:

```
LOOP  
statement1;  
EXIT [ WHEN Condition];  
END LOOP;
```

WHILE LOOP

Syntax:

```
WHILE condition LOOP  
statement1;  
statement2;  
END LOOP;
```

FOR LOOP

Syntax:

```
FOR counter IN [REVERSE]  
LowerBound..UpperBound  
LOOP  
statement1;  
statement2;  
END LOOP;
```

Cursors

Oracle creates a memory area, known as context area, for processing an SQL statement, which contains all information needed for processing the statement, for example, number of rows processed, etc.

A cursor is a pointer to this context area. PL/SQL controls the context area through a cursor. A cursor holds the rows (one or more) returned by a SQL statement. The set of rows the cursor holds is referred to as the **active set**.

A cursor can be named so that it could be referred to in a program to fetch and process the rows returned by the SQL statement, one at a time. There are two types of cursors:

- Implicit cursors
- Explicit cursors

Implicit Cursors

Implicit cursors are automatically created by Oracle whenever an SQL statement is executed, when there is no explicit cursor for the statement. Whenever a DML statement (INSERT, UPDATE and DELETE) is issued, an implicit cursor is associated with this statement. In PL/SQL, you can refer to the most recent implicit cursor as the **SQL cursor**, which always has the attributes like %FOUND, %ISOPEN, %NOTFOUND, and %ROWCOUNT. Any SQL cursor attribute will be accessed as **sql%attribute_name**.

Explicit Cursors

Explicit cursors are programmer defined cursors for gaining more control over the **context area**. An explicit cursor should be defined in the declaration section of the PL/SQL Block. It is created on a SELECT Statement which returns more than one row.

Syntax:

CURSOR CursorName **IS select** statement;

Functions	Syntax
Opening a cursor	OPEN Cursorname
Fetching a record from the cursor	FETCH Cursorname into variable1, variable2,....
Closing a cursor	CLOSE Cursorname

7.3 Programs

1. Write a PL/SQL program to print first 5 department numbers

2. Write a PL/SQL block to accept employee number and display Employee Name, Department name, salary of employees in the format – 'RAVI works in Marketing department and draws 32000/- as salary'.
3. Create a Table EMPSAL with fields-Empno, Empname, Sal, HRA, DA, Gross Salary, PF, Net Salary (assume appropriate datatype and size).

Write a PL/SQL block to accept an employee number existing in EMP table and calculate HRA, DA, Gross Salary, PF, Net_Salary of that employee. Insert the empno, empname, Sal, HRA, DA, Gross Salary, PF, Net Salary into the table EMPSAL:

Use the following formula to calculate salary components-

HRA=50% of Sal

DA=20% of Sal

PF=12% of Sal.

Gross_sal= Sal+ HRA+DA

Net_Sal= Gross_sal-PF

4. Write a cursor in PL/SQL to display department wise employee list.
5. Write a cursor PL/SQL block to allocate incentives to employees and to the corresponding departments handling a project from the Prj_Fund reserved. According to the Prj_Credits a department is having on the Project, a portion of Prj_Fund is given to the department on that project. E.g. Credit is 2 , allocate 20%. Corresponding department has to share 70% of amount received among all employees who are working on that project equally. Remaining 30% is added to Dept_Budget.
6. Write a PL/SQL block using a cursor FOR LOOP to list all projects and their associated tasks.
7. Write a parameterized cursor to display employee with the user given parameters- job and deptno (Using Cursor for loop)
8. Write a parameterized cursor to display first two highest paid employees details (Name, Salary, Department Name) working on a project. The Prj_Id is the user given parameter.

LAB 8 : TRIGGERS

8.1 Objective

To handle runtime errors using predefined and user-defined exceptions, and to automate actions using database triggers.

8.2 Notes

An error condition during a program execution is called an exception in PL/SQL. PL/SQL supports programmers to catch such conditions using **EXCEPTION** block in the program and an appropriate action is taken against the error condition. There are two types of exceptions:

- System-defined exceptions
- User-defined exceptions

Syntax for Exception Handling

System-defined exceptions	User-defined exceptions
<pre>DECLARE <declarations section> BEGIN <executable command(s)> EXCEPTION <exception handling goes here > WHEN exception1 THEN exception1-handling-statements WHEN exception2 THEN exception2-handling-statements WHEN exception3 THEN exception3-handling-statements WHEN others THEN exception3-handling-statements END;</pre>	<pre>DECLARE <exception_name> EXCEPTION; BEGIN IF condition THEN RAISE <exception_name>; END IF; EXCEPTION WHEN <exception_name> THEN statement; END;</pre>

A trigger is a PL/SQL block or a PL/SQL procedure that executes implicitly whenever a particular event takes place. It can either be:

1. Application trigger: Fires whenever an event occurs with a particular application.
2. Database Trigger: Fires whenever a data event (such as DML) occurs on a schema or database.

Syntax:

```
CREATE [OR REPLACE] TRIGGER trigger_name{BEFORE|AFTER|INSTEAD OF}
triggering_event [referencing_clause] ON {table_name | view_name}[WHEN condition]
[FOR EACH ROW] [DECLARE Declaration statements]
BEGIN
```

Executable statements

[EXCEPTION

Exception-handling statements]

END;

The trigger body must have at least the executable section.

The declarative and exception-handling sections are optional.

When there is a declarative section, the trigger body must start with the DECLARE keyword.

The WHEN clause specifies the condition under which a trigger should fire.

NOTE:

- **Exception-handling order matters:** Oracle checks WHEN clauses top-to-bottom and executes only the first match, so specific exceptions should be listed before the generic WHEN OTHERS.
- **Common predefined exceptions:** NO_DATA_FOUND, TOO_MANY_ROWS, DUP_VAL_ON_INDEX, VALUE_ERROR, ZERO_DIVIDE, INVALID_NUMBER — each corresponds to a specific Oracle error code and does not need to be declared.
- **User-defined exceptions** must be declared (e_name EXCEPTION;), explicitly RAISED, and then caught by name — they are not tied to any Oracle error code unless linked with PRAGMA EXCEPTION_INIT.
- **RAISE_APPLICATION_ERROR(-20000 to -20999, message)** is the standard way to signal a custom business-rule failure back to the calling application with a meaningful message.
- **Row-level triggers** (FOR EACH ROW) fire once per affected row and can access :NEW/:OLD; **statement-level triggers** (no FOR EACH ROW) fire once per statement regardless of how many rows are affected, and cannot access :NEW/:OLD.
- **:OLD is available for UPDATE/DELETE, :NEW is available for INSERT/UPDATE** — using :NEW on a DELETE trigger (or :OLD on an INSERT trigger) raises an error since those values don't exist in that context.
- Triggers execute automatically and invisibly on the triggering DML — this is powerful for enforcing rules and auditing, but overuse can make application behaviour hard to trace.

8.3 PROGRAMS

1. Write a PL/SQL program to demonstrate predefined exceptions- INVALID_NUMBER, TOO_MANY_ROWS, NO-DATA_FOUND. Do following operations in the PL/SQL block and handle pre-defined exceptions correspondingly by displaying proper messages.
 - i. Convert ename into number using to_number() function
 - ii. Display ename of employee corresponding to user entered empno. Handle exception if entered empno does not exist.
 - iii. Display ename of employee corresponding to user entered deptno. Handle exception.

2. Write a PL/SQL block to accept, Principle, Interest rate and duration (in years) to calculate Interest to be paid. Handle the exceptions if Principle ≤ 1000 , interest rate < 5 , year < 1 and display proper error message for each.
3. Write a PL/SQL block to accept employee number from user and display employee details such as Empno, Name, and Sal. Handle the exception raised through user defined messages –
 - i. If user entered a non-existing employee number.
 - ii. If the salary more than 25000/-
 - a. If employee exists and salary is less than 25000/- then update that salary to 25000/-
4. Write a PL/SQL trigger to fire when there is an updation of salary of any employee and record the Empno, Dept. Name and Old Salary, date on which salary is modified and user who modified information in the table SAL_MOD (Empno, Dname, Old_Sal, Mod_Date, Modifier).
5. Write a trigger to be invoked when a department on a project updates Prj_fund by adding additional fund. Depending on project duration lapsed, display the message for additional fund for which department is eligible on that project. The project duration lapsed is calculated as difference between current_date (user entered) and start_date. Additional fund eligible is calculated according to the following criteria:

If project duration lapsed is less than 50% then display- You are eligible for Rs. xxxx (xxxx is 70% of additional fund requested) else display- You are eligible for Rs. xxxx (xxxx is 40% of additional fund requested).
6. Create a trigger that ensures an employee's salary cannot be reduced. If an update attempt is made to reduce the salary, the trigger should prevent it and raise an error.
7. Create a trigger that cascades the log of removed dependents.
8. Create a trigger to prevent salary decrease.

LAB 9: PROCEDURES AND FUNCTIONS

9.1 Objective

To create and invoke stored procedures and functions

9.2 Notes

A procedure is a block that can take parameters (sometimes referred to as arguments) and be invoked.

Creating a Procedure

A procedure is created with the CREATE OR REPLACE PROCEDURE statement. The simplified syntax for the CREATE OR REPLACE PROCEDURE statement is as follows:

```
CREATE [OR REPLACE] PROCEDURE procedure_name
[(parameter_name [IN | OUT | IN OUT] type [, ...])]
{IS | AS}
BEGIN
    < procedure_body >
END procedure_name;
```

Creating a Function

A standalone function is created using the CREATE FUNCTION statement. The simplified syntax for the CREATE OR REPLACE PROCEDURE statement is as follows:

```
CREATE [OR REPLACE] FUNCTION function_name
[(parameter_name [IN | OUT | IN OUT] type [, ...])]
RETURN return_datatype
{IS | AS}
BEGIN
    < function_body >
END [function_name];
```

- **Procedure vs function:** a function must RETURN exactly one value and can be called directly from within a SELECT (as in FUNC_ANNUAL_SAL(EMPNO)); a procedure does not return a value through its name and can only be invoked as a standalone PL/SQL statement (BEGIN PROC_NAME(...); END;), never inside a SELECT.
- **Parameter modes:** IN (default) passes a value in, read-only inside the subprogram; OUT returns a value to the caller (starts as NULL inside the subprogram); IN OUT both receives and returns a value through the same parameter

9.3 PROGRAMS

1. Write a procedure to calculate age and service experience of all employees and return these values to calling PL/SQL block and display.
2. Write a procedure to take department name as input to display project being handled by the department and name of the employees working under those projects belonging to the department.
3. Write a PL/SQL function to get the status of a project based on the completion status of its tasks.
4. Write a function to find total project fund on different projects that every department has received. In main program call function for every deptno fetched from Dept. Display following message format for every deptno in main program- (use cursor)

Marketing department has received Rs. 1600000/-
5. Write a stored procedure to display department total salary.

LAB 10 & 11: MINI PROJECT IMPLEMENTATION AND DEMO

For a given problem, the following have to be done:

- Synopsis
- Software Requirement Specifications
- ER Diagram
- Schema Diagram
- Implementation using the concepts studied in the lab

Lab 12 - END TERM PRACTICAL EXAMINATION